



Project Overview

This shared library serves as the foundational backbone for our microservices ecosystem. It encapsulates reusable domain models, utility classes, validation groups, and robust error-handling mechanisms to ensure consistency, eliminate code duplication, and enforce best practices across both Servlet and Reactive microservices.

Technology Stack & Compatibility

- **Java Version:** 17
- **Spring Boot Version:** 3.5.5
- **Build Tool:** Maven
- **Compatibility Matrix:** Fully compatible with Spring Boot 3.5.x and Java 17+

Key Features

Zero-Touch Activation & Configuration

- **config:** Holds internal `@AutoConfiguration` files targeting specific runtime profiles.
- **Zero-Touch Activation:** Automatically wires up beans depending on the host application's environment without requiring manual bean declarations.
- **properties:** Provides resilient, type-safe configuration classes utilizing `Java 17 RECORDS` for modern, immutable property mapping.

Stack Support & Exception Handling

- **Dual-Stack Support:** Offers specialized conditional configurations tailored for both MVC (Servlet) and WebFlux (Reactive) environments.
- **exceptions:** Defines a core operational hierarchy containing abstract base exceptions to ensure unified error structures.
- **handlers:** Features target-specific global exception interceptors that dynamically adapt to the active stack (Servlet vs. Reactive).

Reusable Domains & Utils

- **Reusable Domains & Utils:** Centralizes common data models and high-efficiency utility classes shared across the ecosystem.
- **domains:** Houses shared entities, DTOs, and global models to maintain data contract consistency across multiple microservices.
- **utils:** Delivers high-performance, thread-safe utility classes optimized for shared ecosystem tasks.

Advanced Validation

- **Advanced Validation:** Implements pre-configured validation groups for fine-grained request lifecycle control.
- **validation:** Contains custom validation markers, constraints, and validation groups to enforce robust data integrity at the edge.

Developer Authentication Setup

Before consuming or deploying this library, you must authenticate Maven or Gradle with GitHub Packages using a Personal Access Token (PAT).

Generate Your Token

- In the GitHub UI, navigate to: GitHub Settings → Developer Settings → Personal Access Tokens → Tokens (classic).
- Click Generate new token (classic).
- Select the write:packages and read:packages scopes.
- Copy the generated token (ghp_XXXXXXXXXX).

For Maven project: Copy the token into global `~/.m2/settings.xml` file:

settings.xml

```
<settings>
  <servers>
    <server>
      <id>github</id>
      <username>YOUR_GITHUB_USERNAME</username>
      <password>YOUR_TOKEN_HERE_ghp_XXXXXXXXXX</password>
    </server>
  </servers>
</settings>
```

For Gradle project: Copy the token into global `~/.gradle/gradle.properties` file:

gradle.properties

```
gpr.user=YOUR_GITHUB_USERNAME
gpr.key=YOUR_TOKEN_HERE_ghp_XXXXXXXXXX
```

Deployment

To publish this library to private package registry,

For Maven, add `maven-deploy-plugin` plugin and configure `distributionManagement` inside `pom.xml`

```
<distributionManagement>
  <repository>
    <id>github</id>
    <name>GitHub Packages</name>
    <url>https://maven.pkg.github.com/Kranthi0307/shared-service</url>
  </repository>
</distributionManagement>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-deploy-plugin</artifactId>
      <version>3.1.1</version>
    </plugin>
  </plugins>
</build>
```

For Gradle, add **maven-publish** plugin and configure the **publishing** block inside **build.gradle**

```
plugins {
    id 'maven-publish'
    id 'java'
}

publishing {
    publications {
        mavenJava(MavenPublication) {
            from components.java
        }
    }
    repositories {
        maven {
            name = "GitHubPackages"
            url = uri("https://maven.pkg.github.com/Kranthi0307/shared-service")
            credentials {
                username = project.findProperty("gpr.user")
                password = project.findProperty("gpr.key")
            }
        }
    }
}
```

Run the following command to deploy securely via your CI/CD pipeline:

For Maven,

```
mvn clean deploy
```

For Gradle,

```
gradle clean publish
```

[!WARNING] Update the version before deploying.

Installation

Integrating the library is seamless. Simply add the following dependency to your hosting application:

[!IMPORTANT] Authentication and the source of library needs to be configured

For Maven

```
<repositories>
  <repository>
    <id>github</id>
    <url>https://maven.pkg.github.com/Kranthi0307/shared-service</url>
  </repository>
</repositories>

<dependency>
  <groupId>com.kranthi</groupId>
  <artifactId>shared-service</artifactId>
  <version>1.x.x</version>
</dependency>
```

For Gradle

```
repositories {
  mavenCentral()
  maven {
    url = uri("https://maven.pkg.github.com/Kranthi0307/shared-service")
    credentials {
      username = project.findProperty("gpr.user")
      password = project.findProperty("gpr.key")
    }
  }
}

implementation 'com.kranthi:shared-service:1.x.x'
```

[!NOTE] No additional annotations (like `@Enable*`) or explicit configuration scans are required. The library auto-configures itself upon building the application.

Prerequisites & Configuration

To provide compilation-time IDE support (like autocomplete for configuration properties) in hosting applications, ensure the `spring-boot-configuration-processor` is included in your build.

Intelligent Defaults: The library utilizes `@ConfigurationProperties` bundled with `@DefaultValue` to provide plug-and-play defaults (e.g., `allowedEndpoints="/actuator/health"`) without polluting the host application's properties file.

```
@ConfigurationProperties(prefix = "config.web")
public record MyWebProperties(
    @DefaultValue("/actuator/health")
    String[] publicEndpoints
) {
}
```

Project Structure

```
|—config/ #Framework AutoConfigurations
|—domains/ #Reusable domains, DTOs, and common models
|—exceptions/ #Abstract ecosystem RuntimeExceptions
|—handlers/ #WebMVC/WebFlux @RestControllerAdvice instances
|—properties/ #Java 17 configuration records
|—utils/ #Common thread-safe helper utilities
|—validation/ #Validation groups (CreateValidation, UpdateValidation)
```

Reference Implementation Examples

Validation Groups

```
public interface CreateValidation {}
public interface UpdateValidation {}
```

Domain

```
public record ResponseRecord<T>(String message,
                                T data) {
}
```

Exception

```
public abstract class BaseException extends RuntimeException {

    private final String errorCode;

    protected BaseException(String message, String errorCode) {
        super(message);
        this.errorCode = errorCode;
    }

    protected BaseException(String message, String errorCode, Throwable cause) {
        super(message, cause);
        this.errorCode = errorCode;
    }

    public String getErrorCode() {
        return errorCode;
    }
}
```

Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BaseException.class)
    public final String handleBaseException(BaseException exception) {
        //My exceptions Logic
        return exception.getMessage();
    }
}
```

Error Handling

The library provides a structured, centralized error-handling architecture that standardizes API error responses ecosystem-wide without capturing unintended framework or generic runtime exceptions.

- **Ecosystem Base Exception:** An abstract `BaseException` class that extends `RuntimeException`. All custom business exceptions across your microservices should inherit from this class.
- **Targeted Interception:** Instead of blindly catching generic Java `Exception.class` instances, the global exception handlers explicitly target your base domain exception:

```
@ExceptionHandler(BaseException.class)
```

- **Context Aware:** The library automatically provisions separate `@RestControllerAdvice` implementations tailored for `SERVLET` and `REACTIVE` applications, ensuring appropriate response serialization regardless of the underlying web stack.

[!CAUTION] **Crucial: Bean Naming Collisions**

Since this library provisions beans automatically via `@AutoConfiguration`, you must prevent duplicate bean names between different stack variants (Servlet vs. Reactive).

If a hosting application attempts to build its own bean with a conflicting name (e.g., a generic `globalExceptionHandler`), Spring Boot's context initialization will fail.

Rule: Always namespace your auto-configured internal beans clearly (e.g., `servletGlobalExceptionHandler` vs `reactiveGlobalExceptionHandler`) within your conditional configuration classes.

Local Development

To make changes to this library and test them locally in a hosting application:

- Clone the repository.
- Build and publish the artifact to your local .m2 repository:

```
mvn clean install
```

- Update the version in your hosting application if necessary, and re-import dependencies.

Decision Log (Architecture Decision Records)

ADR 1: Decoupled Architecture via Optional Dependencies

- **Context:** We need to keep this core library lightweight and decoupled from rigid transitive dependency chains.
- **Decision:** Used `<optional>true</optional>` for non-mandatory core dependencies. Hosting applications retain full control over their dependency versions without pollution.

ADR 2: Avoided `@PropertySource` for Default Configurations

- **Context:** Loading properties via `@PropertySource` is a known anti-pattern in modern Spring Boot ecosystems.
- **Decision:** Avoided `@PropertySource` entirely. It breaks smooth integration with Java 17 configuration record types and messes with Spring Boot's natural property loading order. Instead, we lean on `@ConfigurationProperties` with `@DefaultValue`.

ADR 3: Auto-Configuration Split (Servlet vs. Reactive)

- **Context:** The library must safely support both WebMVC (Servlet) and WebFlux (Reactive) microservices without causing runtime bean definition conflicts or forcing developers to manually configure their stack type.

- **Decision:** I made it easy for both Servlet and Reactive environments to work smoothly out of the box by leveraging Spring Boot's modern auto-configuration loading mechanism. The auto-configurations are registered via: `resources/META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. Inside the configuration classes, beans are dynamically and cleanly isolated using conditional boundaries based on the host application's runtime environment:

- `@ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.SERVLET)`
- `@ConditionalOnWebApplication(type = ConditionalOnWebApplication.Type.REACTIVE)`

This ensures that hosting applications automatically get the correct bean implementations (such as the appropriate `GlobalExceptionHandler`) with zero manual setup.

Troubleshooting

- **Properties not auto-completing in IDE:** Ensure the host application has the `spring-boot-configuration-processor` plugin active and your IDE's annotation processing is enabled.
- **Bean conflicts between MVC and WebFlux:** Ensure you haven't mixed both reactive and servlet dependencies on the host application's classpath, which might confuse the conditional auto-configurations.
- **Stale code or version mismatch when switching versions:** If you are switching from a newer version of the library back to an older version and your hosting application is picking up the wrong code, your local `.m2 cache` is likely polluted. Do not delete your entire `.m2 folder`. Instead, delete only this specific library's cache by navigating to `~/.m2/repository/` and deleting the specific library or version folder. After deleting, re-import the dependencies in your hosting application to pull the correct version.

Future Enhancements

The following capabilities are planned for upcoming releases:

- Global Security Configurations & Filters
- Centralized OpenAPI/Swagger documentation configurations
- Standardized Jackson Customizer/Object Mapper configurations
- Multi-Tenant Context Propagation using ThreadLocal (and reactive context equivalents)